

P3 Cholesky solve

Bohaz

25.09.2026

1

In the previous P2 note, we have shown that a Hermitian positive semi-definite matrix can be factorised into a lower triangular matrix multiplied by its adjoint. In this note, we wish to solve the matrix equation

$$A\mathbf{x} = \mathbf{b} \quad (1)$$

where A is a real symmetric $N \times N$ matrix (known), positive definite matrix, which has the following factorisation

$$A = LL^T \quad (2)$$

where L is a lower triangular matrix. $\mathbf{b}$ is a known vector and we wish to solve for $\mathbf{x}$. Solving this equation involves 3 steps: 1. Factorise A into lower and upper triangular matrix. 2. Solve $L\mathbf{y} = \mathbf{b}$ 3. Solve $L^T\mathbf{x} = \mathbf{y}$. When a matrix A is a Hermitian positive definite matrix, the Cholesky factorisation users compared with the standard LU decomposition users, enjoys better numerical stability and roughly twice the efficiency, so we choose Cholesky. First consider

$$A_{ij} = \sum_{k=1}^N L_{ik}L_{kj}^T = \sum_{k=1}^N L_{ik}L_{jk} = \sum_{k=1}^j L_{ik}L_{jk} = \sum_{k=1}^{j-1} L_{ik}L_{jk} + L_{ij}L_{jj} \quad (3)$$

where we let $i \geq j$ without the loss of generality since A is symmetric, and used the fact that $L_{jk} = 0$ for $k > j$ (Too far across to the right, past the diagonal) for the second to last equality. For diagonal entry of A we have

$$A_{ii} = \sum_{k=1}^i L_{ik}^2 = \sum_{k=1}^{i-1} L_{ik}^2 + L_{ii}^2 \quad (4)$$

Then a slight rearrangement of Eq.(3) yields

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik}L_{jk} \right) \quad (5)$$

a slight rearrangement of Eq.(4) yields

$$L_{ii} = \left(A_{ii} - \sum_{k=1}^{i-1} L_{ik}^2 \right)^{1/2} \quad (6)$$

to see how this formula works in practice, notice that we immediately have access to $L_{11} = \sqrt{A_{11}}$ from the start, then $L_{21} = A_{21}/L_{11}$, $L_{31} = A_{31}/L_{11}$ etc., then we gain access

to $L_{22} = \sqrt{A_{22} - L_{21}^2}$, $L_{32} = (A_{32} - L_{31}L_{21})/L_{22}$ etc. until we get the whole L matrix. Eq.(5) and (6) can be easily implemented in a few lines of code. Now we move onto solving the much easier matrix equation $L\mathbf{y} = \mathbf{b}$, in matrix notation

$$\sum_{j=1}^N L_{ij}y_j = \sum_{j=1}^i L_{ij}y_j = b_i \quad (7)$$

where we again used the property of the lower triangular matrix $L_{ij} = 0$ for $j > i$. (Too far across to the right, past the diagonal) Then notice that $y_1 = b_1/L_{11}$, then $L_{21}y_1 + L_{22}y_2 = b_2$ but we already know y_1 , so $y_2 = (b_2 - L_{21}y_1)/L_{22}$, then $y_3 = (b_3 - L_{31}y_1 - L_{32}y_2)/L_{33}$ and more generally

$$y_i = \frac{1}{L_{ii}} \left(b_i - \sum_{j=1}^{i-1} L_{ij}y_j \right) \quad (8)$$

so the order of access to $\mathbf{y}$ is $y_1, y_2, y_3, \dots$, this is known as forward substitution. Finally we solve $L^T\mathbf{x} = \mathbf{y}$, since L^T is upper triangular, we deduce the bottom most element first $x_N = y_N/L_{NN}$, then upwards, all the way until x_1 , this is known as back substitution, the general expression is

$$x_i = \frac{1}{L_{ii}} \left(y_i - \sum_{j=i+1}^N L_{ji}x_j \right). \quad (9)$$

We evaluate this expression for $i = N, N-1, \dots, 1$, so every x_j appearing in the sum has already been calculated. Eq.(8) and (9) can be easily implemented in a few lines of code.